CS781 Capstone Project: In-House Legal Intelligence System

Dhruv Vivekanand Shetty
230370
sdhruv23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

A law firm needs a confidential legal NLP stack that runs without external LLM APIs or third-party pretrained weights. We pretrained four 100–125M-parameter models from scratch (decoder-only, encoder–decoder, prefix-LM, sliding-window/GQA) on a 0.97B-token corpus of Indian court and statute text, sampled by source, with our own 32k BPE tokenizer. We then adapted each model separately to the six downstream tasks using self-implemented parameter-efficient methods and distillation. Bidirectional backbones gave the best official score in four of the five understanding tasks, and LoRMA, training 0.2–0.4% of the parameters, gave the best submission in Phases 3–7. The largest remaining losses come from decision rules: a fixed 0.5 threshold halves statute-identification macro-F1 relative to a dev-fitted one, and greedy summaries repeat themselves.

Code: https://github.com/DhruvShetty22/cs781-legal-nlp-capstone

1 Competition Result

Codabench username: d_230370 **Roll number:** 230370

Table 1: Official phase scores and ranks (recorded 9 November 2026).

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore F1 = 0.8294*	86	5*
2	Legal NER	Strict Macro-F1 = 0.4399*	95	5
3	Rhetorical role prediction	Macro-F1 = 0.4373*	105	7 (5 distinct)*
4	Court judgment prediction	Macro-F1 = 0.6371*	110	5*
5	Legal statute identification	Macro-F1 = 0.1651*	132	3*
6	Abstractive summarization	BERTScore F1 = 0.7662*	130	6
7	Prior Case Retrieval (PCR)	Micro-F1@7 = 0.3166	96	5

*Phase 1 includes the course's starter-code output (not counted towards the minimum); Phase 3 counts 7 finished uploads of 5 distinct files; in Phase 5, one upload failed and two were never evaluated. The highest Phase-6 score, 0.7686 (`llama` LoRMA), is not the leaderboard entry. Four submission-to-run matches (three in Phases 1/2, one in Phase 5) were inferred from file times and contents and confirmed by the student († in Table 7).

Total valid Codabench submissions across all phases: 36 finished (33 distinct, excluding the starter-code output). Counting finished, distinct submissions, the minimum of four is met in every phase except Phase 5, where one of six uploads failed and two were never evaluated by Codabench, leaving three scored submissions.

Table 2: Task definitions. All data comes from Exploration-Lab/CS-781-Capstone.

Ph.	Task	Input → output (`prediction.json`)	Primary metric
1	LM pretraining	512-word judgment prompt → 128-word continuation	BERTScore F1
2	L-NER	judgment text → list of `[start, end, label]` character spans, 12 labels	strict Macro-F1
3	Rhetorical roles	list of sentences → one of 13 roles per sentence (CL and IT subsets)	Macro-F1
4	CJPE	full judgment → outcome $\in \{0, 1\}$	Macro-F1
5	Statute ID	case facts → set of statute IDs (multi-label)	Macro-F1
6	Summarization	full judgment → abstractive summary	BERTScore F1
7	Prior-case retrieval	query case + split-specific candidate pool → top-7 candidate IDs	Micro-F1@7

2 Introduction and Problem Description

The firm's brief is to automate routine legal-document work on Indian case law without sending privileged documents to an external service. This rules out commercial LLM APIs and third-party pretrained models, so every model we use descends from language models we pretrained ourselves on the provided corpus. The seven tasks follow the IL-TUR benchmark [1]; Table 2 gives the input, output and metric of each.

Several constraints shaped the design. The Phase-1 model must have 50M–300M parameters and be trained from random initialization on the provided corpus, with no pretrained LMs or external legal corpora. Training ran on free Colab T4 GPUs (16 GB) whose sessions can be killed, so every run checkpoints to a private Hugging Face repository and every Phase-2–7 run has a 60-minute training cap. Phase 2 is scored by strict character-offset span matching, so token offsets must invert exactly. Phase 5 is multi-label over 100 statutes. Phases 1 and 6 are scored with BERTScore [2], with ROUGE [3] and BLEU [4] as secondary metrics. Phases 4–7 have documents far longer than our 1024-token context.

We investigated five questions: (i) which architecture family to pretrain and how each transfers; (ii) how parameter-efficient methods, with LoRMA [5] compulsory, compare with each other and with full fine-tuning on a small from-scratch backbone; (iii) whether distillation into a half-depth student preserves quality; (iv) how to encode long documents under a 512-token chunk budget; and (v) phase-specific choices such as NER decoding, multi-label thresholds, generation decoding and ranking losses.

Scope. Every phase is built end to end (data loading, preprocessing, model, training, local evaluation with the official scorer, submission packaging) in one notebook template, which is rendered into four per-architecture notebooks. All PEFT and KD code is our own PyTorch [6]; we did not use `peft` or `bitsandbytes`. Backbones are Transformers [7] classes built from our own configs and weights. We did not attempt prefix tuning, external or synthetic data, cross-encoder re-ranking, or chaining phases together; each phase fine-tunes its own copy of a Phase-1 model.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

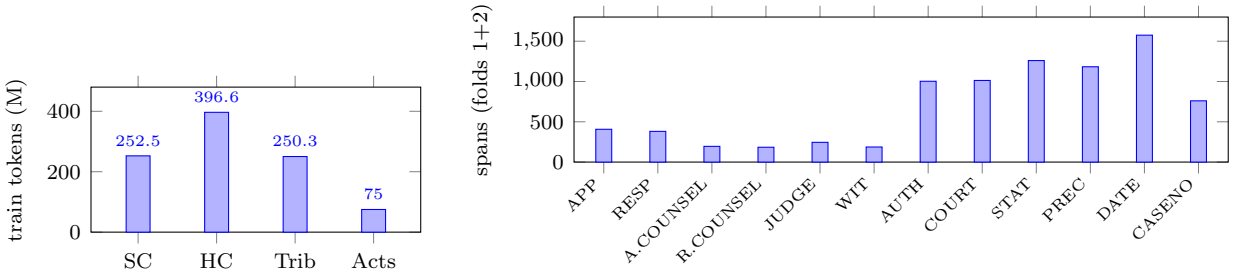
We used only the official Exploration-Lab/CS-781-Capstone dataset and its phase configurations. Table 3 lists the split sizes. Phases 2 and 6 have no official dev split, so we hold out 10 and 300 training documents (seed 42).

3.2 Corpus statistics that actually informed design

Phase 1. The training corpus has 974.4M tokens: 252.5M from the Supreme Court (SC), 396.6M from High Courts (HC), 250.3M from tribunals and 75.0M from Acts (Fig. 1a). We draw training windows from the four sources with probabilities 0.35/0.35/0.15/0.15 (SC/HC/tribunal/Acts) instead

Table 3: Split sizes. Tokens are Phase-1 BPE tokens per training document (median / p90). †Held out from train by us (seed 42); other splits are official.

Ph.	Train	Dev / held-out	Test	Tokens per document
1	974.4M tokens (4 sources)	373 prompts†	1000 prompts	–
2	60 docs (folds 1+2)	10 docs†	35 docs (fold 3)	5533 / –
3	40 CL + 40 IT docs	5 CL + 5 IT docs	5 CL + 5 IT docs	33 / 73 per sentence
4	32,305 docs	994 docs	1517 docs	2479 / 7840
5	42,750 docs	10,181 docs	13,019 docs	638 / –
6	6728 docs	300 docs†	100 docs	3660 / 9102
7	827 queries (pool 4320)	118 queries (pool 1023)	237 queries (pool 1727)	5735 / 15949



(a) Phase 1 tokens per source.

(b) Phase 2 entity spans per label.

Figure 1: Corpus statistics computed by our data-loading code.

of in proportion to their token shares (26/41/26/8%). This up-weights SC and the statutes and down-weights HC and tribunals, so that no single source dominates.

Phase 2. The 70 labelled documents contain 8393 spans over 12 labels, from 184 (R.COUNSEL) to 1575 (DATE); the two counsel labels and WIT have fewer than 200 spans each (Fig. 1b). All spans satisfy `text[s:e]` (end-exclusive), and none is invalid or overlapping. Documents have a median of 26,620 characters (5533 tokens).

Phase 3. The training sets have 10,593 CL and 6252 IT sentences. Fact (2979 CL / 1119 IT) and RatioOfTheDecision (2856 / 1445) dominate. Dissent (284 CL training sentences) and PrecedentOverruled (3 IT training sentences) never occur in dev, which caps the attainable dev macro-F1 at 0.846.

Phase 4. The raw training file has 37,387 records, of which 5082 repeat an ID (none with a conflicting label) and are dropped, leaving 32,305 documents (18,920 label 0, 13,385 label 1). Dev is balanced (497/497). With 8 chunks of 512 tokens, 28.9% of training and 45.5% of dev documents are longer than the budget.

Phase 5. There are 100 statute IDs, each in at least 200 training documents, with 3.70 statutes per document in train (max 31) and 3.92 in dev. The three most frequent IDs cover 27.4%, 24.3% and 18.4% of training documents. Predicting the most frequent statute for every document gives a dev macro-F1 of 0.0045.

Phase 6. Documents have a median of 3129 words and summaries 638. 99.5% of documents exceed 512 tokens, and 76.6% of summaries exceed 480 tokens. On our held-out set the whole document covers 95.3% of the reference unigrams, but the first 512 tokens cover only 39.7% (53.6% at 768).

Phase 7. Queries have 6.42 relevant candidates on average in train (max 52; 10 queries have none) and 8.25 in dev (max 92). Each split has its own pool, and 55–67% of documents exceed the 8-chunk budget. On dev, the oracle top-7 reaches F1@7 of 0.6181, random ranking 0.0074 and our word-level BM25 [8] 0.1123.

Table 4: Phase-1 architectures (all d_{model}=768, vocabulary 32k, tied embeddings).

	`llama`	`prefixlm`	`mistral_swa`	`bart` (v2)
Family	decoder-only	prefix-LM hybrid	efficient (SWA+GQA)	encoder–decoder
Layers	12	12	12	6 enc + 6 dec
Heads (query / KV)	12 / 12	12 / 12	12 / 4	12 / 12
FFN	SwiGLU 2048	SwiGLU 2048	SwiGLU 2048	GELU 3072
Norm / positions	RMSNorm / RoPE	RMSNorm / RoPE	RMSNorm / RoPE	post-LN / learned
Attention	causal	bidir. prefix + causal	causal, window 256	bidir. enc, causal dec
Context	1024	1024	1024	enc $\leq$768, dec 256
Objective	next token	next token after prefix	next token	continuation + infilling
Parameters	109,529,856	109,529,856	100,092,672	125,442,048

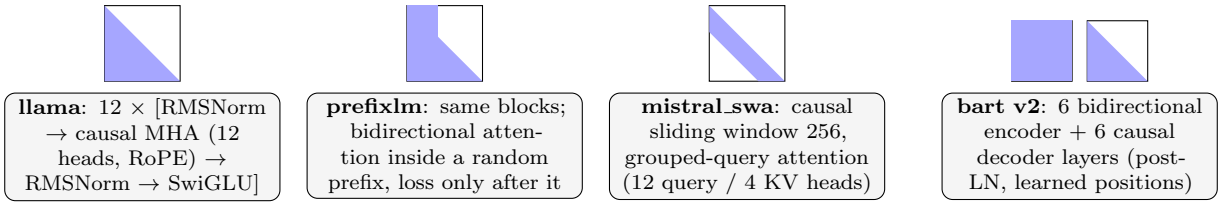


Figure 2: Phase-1 backbones with their self-attention masks (rows: queries; shaded: visible keys). BART shows encoder (left) and decoder (right).

3.3 Preprocessing

Tokenization. Every phase reuses the Phase-1 32k byte-level BPE tokenizer on the original cased text, with punctuation and Unicode unchanged and no special tokens inserted.

NER offsets (Phase 2). Each document is tokenized once with the tokenizer's offset mapping. Token character ranges are trimmed of whitespace, and whitespace-only tokens get empty ranges. A token takes its B-/I- tag (25 BIO tags) from the gold span that covers its first visible character. When decoding, whitespace tokens are transparent, and a span runs from the first to the last visible character of its tokens, so offsets are recovered exactly. Round-tripping the gold spans through this encoding gives a strict macro-F1 of 0.9863, the ceiling of the encoding. We use 512-token windows with stride 256 for training and 128 for prediction, and overlapping predictions are averaged per token.

Sentences (Phase 3). The provided sentence lists are kept unchanged, so there is no mismatch to repair. Each sentence is encoded as its stripped text with a leading space (at most 128 tokens), whole sentences are packed into windows of at most 24 sentences or 512 tokens (stride 12 in training, 6 in prediction, scores averaged over windows), and predictions are written back by index.

Long documents (Phases 4–7). Documents are cut into 512-token chunks plus a final chunk that ends at the document end, and at most 8 evenly spaced chunks are kept (4 in Phase-7 training). Phase 6 instead keeps the document head (512 tokens; 768 for BART) and caps the summary at 480 (512) tokens. In Phase 7 queries and candidates share one encoder and the same chunking, and a query's own ID is excluded from its ranking.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

We pretrained four architecture families from random initialization (Table 4, Fig. 2). All are Hugging Face model classes built from our own configs, with $d_{\text{model}} = 768$ and tied embeddings, and the exact parameter counts of the instantiated models (100.1M–125.4M) lie inside the 50M–300M bound.

Tokenizer. A 32k byte-level BPE [9] trained on 0.4B characters of the corpus (minimum frequency 2). Citations, section symbols and party names get no special handling; byte-level coverage leaves

no input out of vocabulary.

Objectives and data. `llama` [10] and `mistral_swa` [11, 12] predict the next token at all 1024 positions. `prefixlm` [13] draws a prefix length $P \sim \mathcal{U}[64, 896)$, attends bidirectionally inside it and computes the loss only after it. BART [14] encodes part of a window and decodes its continuation.

Training. AdamW [15] ($\beta = (0.9, 0.95)$, weight decay 0.1), peak LR 6×10^{-4} with 300 warm-up steps and cosine decay to 6×10^{-5}, 64 windows of 1024 tokens per step, fp16 and a 3-hour budget per model. The checkpoint with the lowest validation loss (mean over the four sources) is kept. The logged models saw 131–229M tokens (llama 143.9M, prefixlm 130.7M, BART v2 228.7M; the `mistral_swa` log was not kept), i.e. 13–24% of the corpus. Windows are random contiguous spans from sources drawn with the mixing probabilities above.

Decoding. Greedy decoding of 224 new tokens from the last 800 prompt tokens (768 for BART), cut to 128 words. On 373 held-out prompts we compared a repetition penalty of 1.2 with `no_repeat_ngram_size=4`; blocking won for every model, which is why it is used at submission time.

What failed first. An earlier single-notebook pipeline (0.40/0.30/0.20/0.10 mix, 5.5-hour budget) was replaced by the per-architecture template before the logged runs. The first BART run (v1; encodes the last $S \sim \mathcal{U}[128, 768]$ tokens, decodes the next 256) collapsed: its decoder ignored the encoder and returned the same continuation for every prompt. A probe comparing outputs and losses across prompts confirmed it, and we retrained BART as v2 with text infilling (Poisson(3) spans covering 30% of a 512-token window, each replaced by one `<mask>`) on half of the batches and a `<mask>` marker after each continuation prompt. v2's continuation loss rises from 2.85 to 3.58 when it is given another document's prompt, and it produces 8 distinct outputs for 8 prompts, so its decoder now uses the encoder. The fix came after the Phase-1 and Phase-2 deadlines (6 and 11 Oct.). The Phase-1 leaderboard therefore only has BART v1, whose 1000 test outputs are identical, and Phase-2 BART runs could only use v1; Phases 3–7 use v2.

4.2 Transfer protocol (Phases 2–7)

Setup. Every task model starts from a Phase-1 `best/` checkpoint; nothing is trained from scratch. A configuration cell selects the backbone, adaptation method and task options; the four notebooks ran in parallel on four Colab accounts (Fig. 3). Decoder backbones keep their Phase-1 attention pattern. For BART, Phases 2–5 and 7 use the encoder and Phase 6 the full model.

Adaptation methods. All methods are our own PyTorch code (Table 5): LoRA [16], rsLoRA [17], QLoRA [18] with our own NF4 and double quantization, AdaLoRA [19], Houlsby [20] and Pfeiffer [21] adapters, IA^3 [22], prompt tuning [23] (20 soft tokens) and BitFit [24]. The compulsory LoRMA [5] is multiplicative, $h = (I + \frac{\alpha}{r} BA)W_0 x$ with $B = 0$ at initialization, and is merged into W_0 after training. Zero output change at initialization and merge fidelity (logit changes $\leq 10^{-4}$) are checked in the notebooks.

Distillation. KD [25] trains a student that keeps every second Phase-1 block (BART encoder: blocks 0, 2, 4), using the teacher run's saved training-set scores (a sigmoid/BCE version in Phase 5):

$$\mathcal{L}_{\text{KD}} = \tfrac{1}{2}\,\mathcal{L}_{\text{task}} + \tfrac{1}{2}\,T^2\,\text{KL}(\sigma(z_t/T)\,\|\,\sigma(z_s/T))\,, \quad T = 2. \tag{1}$$

Results of the methods. LoRMA at rank 8 trains 0.15–0.44M parameters (0.2–0.4% of the backbone). In Phase 6, prompt tuning (15.4K parameters) comes within 0.003 BERTScore of LoRMA on Codabench, although its held-out loss is clearly worse (2.664 vs. 2.558 for `llama`). In Phase 2, full fine-tuning (0.4399) beats LoRMA with a CRF head (0.4038), but that comparison changes the method and the head at once. Distillation was nearly free in Phases 3 and 4: the

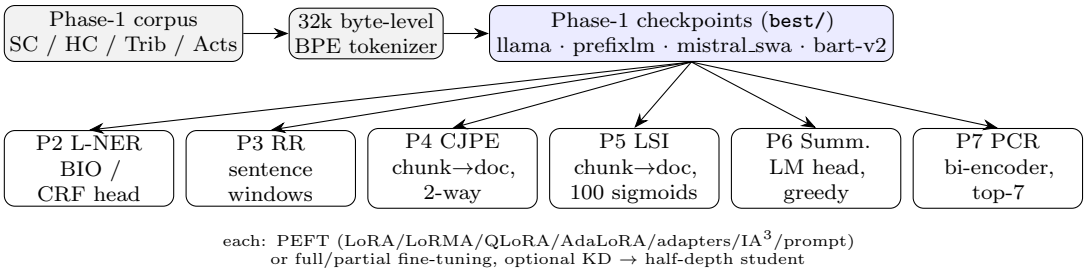


Figure 3: System overview: one tokenizer, four pretrained backbones and six separately fine-tuned task descendants.

Table 5: Adaptation methods implemented (○) and used in at least one submitted run (●). The Phase-5 KD submission was never evaluated.

Ph.	Full	Partial	BitFit	LoRA	rsLoRA	QLoRA	LoRMA	AdaLoRA	Adapter	IA^3	Prompt	KD
2	●	○	○	○			●					○
3	○			○	○		●			○	○	●
4	○	○	○				●	○	○			●
5	○	○				○	●	○	○		○	●
6	○			○		○	●	○	○		●	○
7	○	○				○	●			○	○	○

BART student is within 0.007 of BART LoRMA and, in Phase 4, above its `llama` teacher, after 10.6 instead of 60 training minutes. In Phase 5, where the student must reproduce 100 sigmoid outputs, it lost 0.049 dev macro-F1.

4.3 Phase 2 – Legal NER

We use token classification and reconstruct character offsets (Section 3). The head is dropout 0.1 and a linear layer to 25 BIO tags, optionally followed by our own linear-chain CRF [26] with Viterbi decoding over the whole document. Window-level scores are averaged per token before decoding. Class imbalance is not reweighted (O-tag loss weight 1.0). Decoding reads one tag sequence, so spans cannot overlap; it is either *strict* (only a B- tag opens a span) or *lenient* (an I- tag may also open one). We did not use thresholds or n-best spans. Submitted variants are full fine-tuning of three backbones and `llama` LoRMA with a CRF head. BART could only be fine-tuned from the collapsed v1 checkpoint and produced 39,953 test spans, 7–12 times as many as the other runs.

4.4 Phase 3 – Rhetorical role prediction

A sentence vector is the mean of its token states inside a multi-sentence window, followed by a linear layer over the 13 roles. The window gives every sentence left context and, for bidirectional backbones (prefixlm, BART encoder), right context as well. Classification is per sentence, with no document-level CRF or transition constraints. CL and IT are trained jointly, and model selection uses macro-F1 on the pooled dev set.

4.5 Phase 4 – Court judgment prediction

The model is hierarchical. Each selected 512-token chunk is encoded independently, a chunk vector is the mean of its token states, and the document vector is the mean of the chunk vectors. The head is dropout 0.1 and a 2-way linear layer trained with cross-entropy, and prediction is by argmax. Phase-2/3 structure is not used as features.

4.6 Phase 5 – Legal statute identification

The encoder is the Phase-4 hierarchical one with 100 independent sigmoid outputs trained with binary cross-entropy. The submitted rule is a global threshold of 0.5. The notebooks also fit a global

threshold and per-label thresholds on dev; since these are fitted on the evaluation data we report them only as diagnostics.

4.7 Phase 6 – Abstractive summarization

Decoder-only backbones read the document, a separator line "Summary:", the summary and EOS, with the loss on summary tokens only. The prefix-LM attends bidirectionally over the document, and its generation path is patched so that the prefill is bidirectional too. BART encodes the document followed by `<mask>` and decodes the summary. We use one truncated pass over the document head rather than chunk-and-merge. Decoding is greedy with at most 480 new tokens (512 for BART), repetition penalty 1.0 and no n-gram blocking. No post-training or alignment step was used. Checkpoints are selected by held-out loss.

4.8 Phase 7 – Prior Case Retrieval

A shared-weight bi-encoder embeds queries and candidates: chunk vectors are mean-pooled token states, the document vector is the mean over chunks, and $s(q, d) = \cos(\mathbf{q}, \mathbf{d})/\tau$ with $\tau = 0.05$. Training uses InfoNCE [27] over in-batch candidates plus one random negative per pair, with the query's other positives masked out,

$$\mathcal{L} = -\log \frac{e^{s(q,d^+)}}{e^{s(q,d^+)} + \sum_{d \in \mathcal{N}(q)} e^{s(q,d)}}, \tag{2}$$

or ListNet [28] with a uniform target over the relevant candidates. Long documents use the chunk budget of Section 3. At test time we embed the split's pool once, score each query against it, drop the query's own ID and output the top 7. A word-level Okapi BM25 (k_1=1.2, b=0.75) serves as a reference. BM25 fusion and re-ranking exist but were off in the submitted runs, because BM25 alone reached only 0.1123 dev F1@7, below the untrained Phase-1 encoders (0.24–0.26). A further failure is generalization: BART InfoNCE scored 0.3369 on dev but 0.2942 on test.

5 Experiments and Implementation Details

5.1 Compute and training protocol

Compute. Free Colab T4 GPUs (16 GB), PyTorch 2.11 and Transformers 5.18. Every run saves its configuration, metrics, predictions and checkpoints to a private Hugging Face repository, so killed sessions lose little.

Shared recipe (Phases 2–7). AdamW with weight decay 0.01, head learning rate 10^{-3}, 6% warm-up then linear decay, gradient clipping at 1.0, fp16, seed 42 and a 60-minute training cap. The checkpoint with the best dev score is kept (Phase 6: best held-out loss; Phase 1: lowest validation loss). Table 6 gives the settings of the best submission in each phase.

5.2 Local evaluation

Phases 3, 4, 5 and 7 select models on the official dev split; Phases 2, 6 and 1 use the held-out sets of Table 3. Local scores use code matched to the official evaluation scripts, and each notebook validates the ID set and output schema before packaging. Codabench and local scores agree within 0.006 in Phases 1, 5 and 6 and diverge where the local set is small or differs in kind. In Phase 2 (10 held-out documents) all three logged runs scored 0.05–0.07 higher on the 35 test documents. In Phase 3 (10 dev documents) the ordering of the three logged backbones, which differ by only 0.005 locally, was not preserved on test. In Phase 7 the test pool has 1727 candidates against 1023 in dev, and every run scored lower on test.

Table 6: Hyperparameters of the best-scoring Codabench submission per phase. Params: trainable / total (M). LoRMA is `plus`, r=8, α=16 on `q_proj/v_proj`. †Run inferred from file times. ‡From the logged BART InfoNCE run; no log was kept for the ListNet run, assumed to differ only in the loss. nr-4 = `no_repeat_ngram_size=4`. P6's leaderboard entry is BART LoRMA (0.7662), not this run.

	P1	P2	P3	P4	P5	P6	P7
Backbone	llama	llama	prefixlm	prefixlm	bart	llama	bart
Method	pretrain	full†	LoRMA	LoRMA	LoRMA†	LoRMA	LoRMA+ListNet
Params (M)	109.5	109.5	0.3/109.8	0.3/109.8	0.2/68.2	0.3/109.8	0.15/68.1‡
Context (tok.)	1024	512	512	8×512	8×512	512+480	4×512‡
Peak LR	6e-4	5e-5	5e-4	5e-4	5e-4	5e-4	5e-4‡
Batch × accum.	8×8	16×1	8×1	2×4	2×4	4×4	4 pairs‡
Best at	step 2196	epoch 4	epoch 6	step 1904	step 10688	step 1263	not logged
Dropout	0.0	0.1	0.1	0.1	0.1	–	–
Decision	greedy+nr-4	strict BIO	argmax	$p \geq 0.5$	$p \geq 0.5$	greedy	top 7

5.3 Ablations and controlled comparisons

Each comparison changes one factor; all numbers are in Table 7.

(a) Backbone. At the same LoRMA recipe, Codabench macro-F1 varies by 0.061 in Phase 3 and 0.046 in Phase 4, with `prefixlm` first in both, but only by 0.009 BERTScore in Phase 6.

(b) Adaptation method. Prompt tuning vs. LoRMA, and distilled students vs. their LoRMA teachers: see Section 4.2.

(c) BART objective. Fixing the collapse raised held-out BERTScore from 0.8047 (v1) to 0.8260 (v2) and BLEU-4 from 0.009 to 0.046.

(d) Decoding. n-gram blocking beat a repetition penalty of 1.2 for every Phase-1 model (+0.0085 `llama`, +0.009 `prefixlm`, +0.0001 BART v2).

(e) Threshold. In Phase 5 a global threshold fitted on dev (0.12–0.14) doubles macro-F1 from 0.15–0.16 to 0.29–0.31, and per-label thresholds reach 0.33–0.35. Both are fitted on the evaluation set and so optimistic, but the gain is ten times the dev spread between backbones (0.014).

(f) Ranking loss. For BART in Phase 7, ListNet scored 0.3166 against 0.2942 for InfoNCE on Codabench.

(g) Phase-1 transfer. Without Phase-7 training, the Phase-1 encoders already reach 0.24–0.26 dev F1@7, over twice word-level BM25 (0.11); fine-tuning adds 0.06–0.10. We did not ablate hierarchical encoding against a first-chunk-only model.

6 Results

Table 7 lists every variant with its local and official score. Learning curves are omitted for space; the run logs print every evaluation. Held-out ROUGE-L in Phase 6 stays within 0.182–0.194 across all six variants.

Best system per phase. `llama` won Phases 1 (0.8294), 2 (full fine-tuning, 0.4399) and 6 (LoRMA, 0.7686); `prefixlm` LoRMA won Phases 3 (0.4373) and 4 (0.6371); BART LoRMA won Phases 5 (0.1651) and 7 (ListNet, 0.3166).

Why. Classification and retrieval build one vector from mean-pooled token states. In a causal model the early tokens of a chunk never see the rest of it, while `prefixlm` and the BART encoder let every token use context on both sides. Bidirectionality alone is not enough, though: BART's smaller encoder was last in Phase 3. Generation matches the decoder-only training objective, and `llama` was also the strongest Phase-1 model. In Phase 2 the backbone is confounded with the decoding rule (strict for `llama`, lenient for the others).

Table 7: Per-phase variants: local dev/held-out primary metric ("Local") and Codabench primary metric ("CB"); "–" = no local run log; n/s = not submitted in that phase; n/e = submitted but never evaluated; † = run inferred from file times and contents. Shared settings: P1 greedy + no-repeat-4; P3 sentence windows; P4/P5 8 even chunks, mean pooling, threshold 0.5; P6 head truncation; P7 mean pooling, random negatives.

Ph.	Variant	Local	CB
1	starter-code output†	–	0.8010
1	llama	0.8309	0.8294
1	bart v1 (collapsed)	0.8047	0.8048
1	bart v2	0.8260	n/s
1	prefixlm	0.8272	0.8269
1	mistral_swa†	–	0.8265
2	llama full, 1st run†	–	0.3957
2	llama full, strict†	0.3903	0.4399
2	prefixlm full, lenient	0.3383	0.3893
2	mistral_swa full, lenient†	0.3206	0.3886
2	llama LoRMA + CRF	–	0.4038
3	llama LoRMA	0.4027	0.4118
3	prefixlm LoRMA	0.4033	0.4373
3	mistral_swa LoRMA	0.4073	0.4083
3	bart LoRMA	–	0.3765
3	bart KD	–	0.3697
4	llama LoRMA	0.6012	0.5908
4	bart LoRMA	0.5999	0.6067
4	prefixlm LoRMA	0.6649	0.6371
4	mistral_swa LoRMA	0.6113	0.6034
4	bart KD (llama teacher)	0.5920	0.6017
5	llama LoRMA	0.1542	0.1543
5	bart LoRMA†	0.1591	0.1651
5	mistral_swa LoRMA	0.1454	failed
5	prefixlm LoRMA	–	0.1544
5	bart KD (bart teacher)	0.1101	n/e
6	llama LoRMA (512)	0.7719	0.7686
6	bart LoRMA (768)	0.7693	0.7662
6	prefixlm LoRMA (512)	0.7643	0.7592
6	mistral_swa LoRMA (512)	0.7622	0.7631
6	llama prompt (512)	0.7657	0.7658
6	bart prompt (768)	0.7677	0.7636
7	llama LoRMA InfoNCE	0.3235	0.3154
7	bart LoRMA InfoNCE	0.3369	0.2942
7	prefixlm LoRMA InfoNCE	0.3091	0.3030
7	mistral_swa LoRMA InfoNCE	–	0.3048
7	bart LoRMA ListNet	–	0.3166

Cost. Every LoRMA run fitted in 60 minutes and 3.6 GB of GPU memory. The Phase-4 student trained in 10.6 instead of 60 minutes and predicts 5× faster than its teacher (12.3 s vs. 64.6 s on dev).

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phase 2. On the held-out set, the strict `llama` run has 701 exact matches, 190 right-label-wrong-boundary spans, 184 spurious spans and 479 misses; the lenient `prefixlm` and `mistral_swa` runs have 794/709 exact matches but 590/656 boundary errors, 514/411 spurious spans and only 131/224 misses. Lenient decoding lets any I- tag open a span, so a fragment of a mis-tagged entity becomes an extra span, and boundary and spurious counts rise together. WIT is almost never found (F1 ≤ 0.04, against 0.87–0.90 for COURT): witnesses look like other person names, are told apart only by their role, and have only 187 labelled spans.

Phase 3. The main confusion is between petitioner and respondent arguments (103–124 sentences per run), which use the same vocabulary and differ only in who is speaking, often stated outside a 24-sentence window. CL macro-F1 (0.29–0.34) is far below IT (0.41–0.44).

Phase 4. Predictions swing between classes during training: `prefixlm` predicts 185, 897, 510 and 686 positives out of 994 balanced dev documents after 12, 24, 36 and 48 minutes, and the final `llama` model has recall 0.44 at precision 0.68. With 45.5% of dev documents only partly read, the passage that states the outcome can be missed.

Phase 5. At threshold 0.5, 36–42% of dev documents get no statute at all (3645–4248 of 10,181), although each has 3.92 on average, and only 51–79 of the 100 statutes are ever predicted on test. Most statutes are positive in only a few per cent of documents, so BCE keeps their probabilities below 0.5; the dev-fitted thresholds of 0.12–0.14 show where the boundary actually lies.

Phase 6. Summaries average 411–449 words against 866 for the references, and 91–95% of their trigrams are repeats (references: 9.9%), because greedy decoding has no repetition control. A `llama`

LoRMA output begins "The appellant was a contractor of the appellant company. The appellant was a contractor of the appellant company." The head-only input covers 39.7% of reference unigrams.

Phase 7. 16–18 of 118 dev queries have no relevant case in the top 7 (BM25: 57). One mean-pooled vector over at most 8 chunks must summarize documents of which 55–67% exceed that budget.

7.2 What the metrics do not measure

BERTScore has a high floor on legal text: BART v1, which returns the same text for all 1000 Phase-1 prompts, scores 0.8048, only 0.025 below our best model, and Phase-6 summaries made mostly of repeats still score 0.76–0.77. Neither ROUGE-L nor BERTScore checks whether a summary states the actual holding or whether a predicted outcome follows from the reasoning. Strict span-F1 counts an entity with a one-character boundary error as both a false positive and a false negative, so it rewards offset perfection rather than typing quality. Macro-F1 weighs a statute cited in under 1% of cases like one cited in 27%, and Micro-F1@7 fixes the cutoff at seven although dev queries have 8.25 relevant cases on average, so even a perfect ranking scores only 0.62. For a firm, a high leaderboard score could therefore still hide summaries that miss the holding.

7.3 Limitations and what you would do next

The logged Phase-1 models saw only 13–24% of the corpus in their 3-hour budget. Every result is a single run (seed 42), Phases 2 and 3 select models on 10 documents, and there was no human legal evaluation. Phases were trained separately, so errors do not propagate, but no phase helps another either. Next, in order of expected gain: a Phase-5 threshold fitted on a split held out from training; n-gram blocking and wider input coverage for Phase 6; at least one full epoch of Phase-1 pretraining; always including the final chunk in Phase 4; and mined hard negatives in Phase 7.

7.4 Surprising findings

Untrained Phase-1 encoders retrieve prior cases twice as well as BM25 (0.24–0.26 vs. 0.11 dev F1@7), although we expected lexical matching to be strong on long legal documents. A prompt-independent output scored within 0.025 BERTScore of our best Phase-1 model. The three-block Phase-4 student scored 0.6017 on Codabench against 0.5908 for its full-depth `llama` teacher.

8 Conclusion

Three choices moved the metrics: bidirectional context in the classification phases (+0.026 to +0.046 macro-F1 for `prefixlm`), repairing BART's objective (+0.021 BERTScore), and decision rules, namely strict decoding in Phase 2, n-gram blocking in Phase 1 and, above all, the Phase-5 threshold. The choice of adaptation method (prompt tuning vs. LoRMA, at most 0.003) and distillation in Phases 3–4 (at most 0.007) did not. The main lesson is that, under a fixed parameter budget and a single corpus, a LoRMA update of 0.2–0.4% of the weights is enough to carry a from-scratch 110M legal LM into every task, and that the remaining gains lie in what the backbone can see and in how its outputs become decisions.

LLM Usage Declaration

The answers below were drafted with Claude from the project history and checked by the student.

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor.

Answer: Claude models (Anthropic), used through the Claude Code tool throughout the project; Claude Opus 5.5 wrote this report and the repository documentation.

Q2. For each row, mark Yes or No and state the purpose in one short phrase.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	explanations of the methods (LoRMA, QLoRA/NF4, prefix-LM masks, SWA); explanatory reference implementations of the four architecture families, not used for training
Ideation / brainstorming at a high level	Yes	discussing design options, which were implemented as configuration switches
Debugging small ancillary snippets	Yes	notebook errors on Colab; the BART v1 diagnostic cells
Plotting / visualization code	Yes	pgfplots/TikZ code for Figs. 1, 2 and 3
Grammar / writing style of the report	Yes	the report text was written by Claude
Report structure / section wording	Yes	all sections drafted from the code, run logs and Codabench records
Designing the model architecture	Yes	configuration code for the four Phase-1 families (Hugging Face classes) and the task heads; the families follow capstone §7.3
Hyperparameter search / tuning	No	no values were chosen from results by Claude; the default values in the configuration cells were written by Claude with the templates, and the student chose which variants to run
Analysis of results / conclusions	Yes	Sections 4.2 (results of the methods), 4.8, 5.2, 5.3, 6, 7 and 8 and the abstract's findings were drafted by Claude from the logged numbers at the student's request; the student has asked the instructor for leniency on this point
Writing the core training / inference code	Yes	the per-phase notebook templates (data pipelines, model wrappers, PEFT and KD implementations, training, evaluation, submission) were written by Claude at the student's direction; the student ran and debugged them on Colab
Other (specify)	Yes	repository READMEs; matching Codabench submissions to runs

Q3. Did you refer to sources other than lecture slides and an LLM?

Answer: The capstone document, the official starter notebook and the official evaluation scripts, the IL-TUR paper [1], and the method papers cited in the text.

Q4. Apart from any permitted LLM use, what was *your* contribution?

I ran every training and inference job across four Colab accounts. I chose which variants to run in each phase (the run names record them: architectures, LoRMA vs. prompt tuning, a CRF head

and strict decoding in Phase 2, distillation teachers, the ListNet loss in Phase 7). I also made and archived all Codabench submissions. Two things I tried failed. First, the continuation-only BART (v1) collapsed: all 1000 of its Phase-1 test continuations were the same text. I concluded that an encoder–decoder trained only to continue text can lower its loss through the decoder alone, and retraining it with text infilling and a mask marker made its outputs depend on the prompt; because the fix came after the Phase-2 deadline, BART never had a working Phase-1 or Phase-2 entry. Second, the fixed 0.5 threshold in Phase 5 left more than a third of dev documents without any statute, which showed me that with 100 rare labels the decision rule matters as much as the model. Phase-6 summaries also degenerated into repetition under greedy decoding, which BERTScore barely penalized. Across phases I learned that bidirectional context helped the understanding tasks and that a low-rank LoRMA update was enough to transfer the from-scratch models to every task.

References

[1] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for Indian legal text understanding and reasoning," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[2] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[3] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.

[4] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proceedings of ACL*, pp. 311–318, 2002.

[5] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[6] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[7] T. Wolf, L. Debut, V. Sanh, *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of EMNLP: System Demonstrations*, 2020.

[8] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[9] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of ACL*, 2016.

[10] H. Touvron, T. Lavril, G. Izacard, *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[11] A. Q. Jiang, A. Sablayrolles, A. Mensch, *et al.*, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.

[12] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of EMNLP*, 2023.

[13] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[14] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," in *Proceedings of ACL*, 2020.

[15] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[16] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[17] D. Kalajdzievski, "A rank stabilization scaling factor for fine-tuning with LoRA," *arXiv preprint arXiv:2312.03732*, 2023.

[18] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, 2023.

[19] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "Adaptive budget allocation for parameter-efficient fine-tuning," in *International Conference on Learning Representations*, 2023.

[20] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *International Conference on Machine Learning*, 2019.

[21] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, and I. Gurevych, "AdapterFusion: Non-destructive task composition for transfer learning," in *Proceedings of EACL*, 2021.

[22] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems*, 2022.

[23] B. Lester, R. Al-Rfou, and N. Constant, "The power of scale for parameter-efficient prompt tuning," in *Proceedings of EMNLP*, 2021.

[24] E. Ben Zaken, Y. Goldberg, and S. Ravfogel, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of ACL (Short Papers)*, 2022.

[25] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[26] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *International Conference on Machine Learning*, 2001.

[27] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[28] Z. Cao, T. Qin, T.-Y. Liu, M.-F. Tsai, and H. Li, "Learning to rank: From pairwise approach to listwise approach," in *International Conference on Machine Learning*, 2007.